A

Mini Project Report

On

PERSONAL EXPENSE TRACKER

For

Partial fulfillment of award of the

B.Tech. Degree

in

Computer Science and Information Technology



2026-27

Under the supervision of

Pragya Siddhi

[Supervisor designation to be verified]

Team Members

Prakhar Kumar Singh (2501920110264)
Prakhar Srivastava (2501920110265)
Prakash Yadav (2501920110263)

Department of Computer Science and Information Technology

G. L. Bajaj Institute of Technology and Management
Plot No 2, Knowledge Park-III, Greater Noida-201306
Dr. A.P.J. Abdul Kalam Technical University, Uttar Pradesh
2026

DECLARATION

We hereby declare that the mini project report entitled **"Personal Expense Tracker"**, submitted in partial fulfilment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Information Technology, is a record of the work carried out by our team under the supervision of Pragya Siddhi at G. L. Bajaj Institute of Technology and Management, Greater Noida.

We have prepared this report using the project files and available application screenshots. Any external ideas, documentation, or technical material used in the report are acknowledged through citations and references. We understand that the report must represent the actual work completed by the team and that claims about testing, performance, or functionality should be supported by evidence.

We further declare that this report has not been submitted, in whole or in part, for the award of another degree or qualification. The final declaration should be signed only after the team has reviewed the report and confirmed that all statements accurately represent the submitted project.

Student	Signature	Date
Prakhar Kumar Singh		
Prakhar Srivastava		
Prakash Yadav		

Place: Greater Noida

GL BAJAJ
Institute of Technology & Management

Department of Computer Science and Information Technology

CERTIFICATE

This is to certify that the mini project report entitled **"Personal Expense Tracker"** has been prepared by Prakhar Kumar Singh (2501920110264), Prakhar Srivastava (2501920110265), and Prakash Yadav (2501920110263), students of the Department of Computer Science and Information Technology, G. L. Bajaj Institute of Technology and Management, Greater Noida, in partial fulfilment of the requirements for the award of the Bachelor of Technology degree.

The work described in this report is based on the project files and interface evidence made available for review. The report is submitted for academic evaluation under the supervision of Pragya Siddhi. The supervisor should verify the wording and sign this certificate only after reviewing the final report and project demonstration.

Project Supervisor	Head of Department
Pragya Siddhi	Dr. P. C. Vashist
Designation: to be verified	Department of CS & IT
Signature: _____	Signature: _____
Date: _____	Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project supervisor, Pragya Siddhi, for guidance and support during the development and documentation of the Personal Expense Tracker. The project provided an opportunity to apply programming, database, graphical user-interface, data-visualisation, and software-testing concepts to a practical problem.

We thank the Department of Computer Science and Information Technology and G. L. Bajaj Institute of Technology and Management for providing an academic environment in which we could work collaboratively and learn through implementation. We also acknowledge the maintainers of Python, Tkinter, SQLite, Matplotlib, and OpenPyXL, whose official documentation supports the technologies used in this project.

Finally, we thank our teammates for their contributions to the project and report preparation. We recognise that a reliable application requires not only implementation but also careful verification, clear documentation, and responsible handling of stored financial information.

ABSTRACT

The Personal Expense Tracker is a desktop application developed to help users record, organise, review, and analyse personal income and expenses. The application is implemented in Python, with Tkinter used to create the graphical user interface and SQLite used for persistent transaction storage. Matplotlib supports graphical summaries, while OpenPyXL is listed as a dependency for Excel workbook handling.

The transaction model stores a date, transaction type, category, amount, payment method, description, automatically generated identifier, and creation timestamp. The dashboard presents overall income, overall expenses, balance, current-month expenses, savings-rate information, a monthly summary, and recent transactions. Separate areas support expense and income management. The reports interface provides filters for date range, transaction type, category, and text search, and the project documentation describes CSV and Excel export. The charts module includes category distribution, monthly income-versus-expense comparison, and monthly expense trend visualisations.

The project aims to make manual record-keeping more organised and to provide a convenient overview of financial activity. The system is a local desktop application; it does not claim cloud synchronisation, user authentication, bank integration, or automated financial advice. Unusual-spending detection, forecasting, and smart budget recommendations are treated as future enhancements rather than completed features. Testing is recorded as partially completed, so final acceptance should be based on repeatable test cases and evidence from the final application.

Keywords: Personal finance, expense tracking, Python, Tkinter, SQLite, Matplotlib, reporting, CSV export, Excel export.

TABLE OF CONTENTS

Declaration ..... **i**

Certificate ..... **ii**

Acknowledgement ..... **iii**

Abstract ..... **iv**

List of Figures ..... **v**

List of Tables ..... **vi**

List of Abbreviations ..... **vii**

Chapter 1: Introduction **1**

Chapter 2: Existing Technologies **7**

Chapter 3: Proposed Work **13**

Chapter 4: Methodology **19**

Chapter 5: Result Analysis **27**

Chapter 6: Conclusion and Future Work **34**

Environment and Sustainability Considerations **37**

References **39**

Appendix A: Verification Checklist **42**

Note: Page numbers are indicative and may shift if the report is edited or reflowed.

LIST OF FIGURES

Figure 3.1. High-level architecture of the Personal Expense Tracker

Figure 3.2. Transaction data flow

Figure 4.1. Development and verification workflow

Figure 5.1. Dashboard interface

Figure 5.2. Expense management interface

Figure 5.3. Income management interface

Figure 5.4. Financial charts interface

Figure 5.5. Reports and filtering interface

LIST OF TABLES

Table 2.1. Comparison of record-keeping approaches

Table 3.1. Functional requirements

Table 3.2. Non-functional requirements

Table 3.3. Module responsibilities

Table 4.1. Transaction data dictionary

Table 4.2. Test cases and expected outcomes

Table 5.1. Evidence-based feature review

Table 5.2. Limitations and risks

LIST OF ABBREVIATIONS

Abbreviation	Meaning
CSV	Comma-Separated Values
DB	Database
GUI	Graphical User Interface
ID	Identifier
SQL	Structured Query Language
SQLite	Embedded SQL database engine
UI	User Interface
CRUD	Create, Read, Update, Delete
IDE	Integrated Development Environment
OS	Operating System
UX	User Experience
API	Application Programming Interface
PDF	Portable Document Format
CS & IT	Computer Science and Information Technology

CHAPTER 1: INTRODUCTION

1.1 Introduction

Personal financial management involves recording income, tracking spending, comparing financial activity across periods, and reviewing the difference between money received and money spent. When records are kept informally in memory, scattered notes, or unstructured files, it becomes difficult to answer basic questions such as how much was spent in a month, which categories account for the largest share of expenses, and whether the current balance is positive or negative.

The Personal Expense Tracker addresses this record-keeping problem through a desktop application. Users can enter income and expense transactions, maintain transaction details, view a dashboard, inspect charts, and filter reports. The application brings common tasks into one interface instead of requiring users to maintain separate lists.

1.2 Background

Spreadsheet tools and paper notebooks are useful for basic tracking, but their usefulness depends on consistent data entry and manual calculation. A purpose-built tracker can standardise fields, calculate totals, store records in a database, and present summaries automatically. The project demonstrates how a small software system can combine a graphical interface, a relational data store, data aggregation, and visualisation.

The application uses Python for application logic. Tkinter provides desktop widgets such as windows, labels, entry fields, buttons, tables, and selection controls. SQLite stores transaction records in a local database file. Matplotlib is used for charts, and OpenPyXL is listed as an external dependency for spreadsheet workbook support. These choices make the application suitable for a small local project without requiring a separate database server.

1.3 Problem Statement

A user who records personal transactions manually may experience incomplete records, inconsistent categories, arithmetic errors, and difficulty identifying spending patterns. Data stored in separate files can be hard to search, update, or summarise. The problem addressed by this project is to provide a simple local application that centralises income and expense records and supports review through totals, charts, filters, and exportable reports.

1.4 Aim

The aim is to design and implement a desktop-based Personal Expense Tracker that allows users to maintain transaction records and review basic personal financial summaries.

1.5 Objectives

• Provide a graphical interface for managing income and expense transactions.

• Store transaction records persistently in an SQLite database.

• Support adding, viewing, editing, and deleting transactions.

• Display high-level totals such as income, expenses, and balance.

• Provide category and monthly visual summaries using charts.

• Allow reports to be filtered by date, type, category, and search text.

• Support export of report data to CSV and Excel as documented in the project.

• Keep the design modular so that future features can be added without replacing the entire application.

1.6 Scope

The current scope is limited to a local desktop application and the features supported by the supplied code and project documentation. The database is a local SQLite file. The application has dedicated modules for the dashboard, expenses, income, charts, and reports. The scope does not include bank-account connections, cloud synchronisation, multiple-user access, authentication, mobile support, or automatic financial advice.

1.7 Motivation and Significance

The project offers practical experience with a common software pattern: capturing user input, validating it, storing it in a database, retrieving records, calculating aggregates, and presenting the result visually. It also illustrates how modular programming can separate database operations from interface screens. From a user perspective, the tracker can make routine record keeping more structured and make monthly review easier.

1.8 Organisation of the Report

Chapter 1 introduces the problem and project objectives. Chapter 2 reviews existing approaches and the technologies used. Chapter 3 describes the proposed system, its requirements, modules, and architecture. Chapter 4 explains the data model, processing flow, implementation method, and testing plan. Chapter 5 presents the available interface evidence and discusses results and limitations. Chapter 6 concludes the work and lists future improvements. An additional section discusses environment and sustainability considerations, followed by references and a verification checklist.

CHAPTER 1: INTRODUCTION (CONTINUED)

1.9 Definitions Used in This Report

Income

Income is money received by the user. In the application, income is represented as a transaction whose type is marked as income and whose amount contributes to the total-income calculation.

Expense

An expense is money spent by the user. Expense transactions are grouped by category for the category chart and included in expense totals and monthly expense summaries.

Balance

The balance displayed by the application is calculated as total income minus total expenses. It is a simple aggregate of the transactions recorded in the database, not a connection to a bank balance.

Savings rate

The dashboard includes a savings-rate indicator. The precise formula and handling of zero income should be checked in the final code and interface before the report is submitted. A common definition is (income minus expenses) divided by income, multiplied by 100, but this report does not claim that formula as verified unless confirmed by the implementation.

1.10 Assumptions

The system assumes that the user enters transaction details accurately and chooses appropriate transaction types and categories. The summaries can only be as accurate as the stored data. The application is assumed to be run on a computer with a compatible Python installation and the required libraries. The report does not assume that the application has been independently audited or deployed for production use.

1.11 Constraints

The local database design is suitable for a single-user academic project but does not provide the controls typically expected of a multi-user financial service. The project does not show an authentication layer, encryption-at-rest feature, automated backup service, or synchronisation mechanism. The interface also depends on the desktop environment and installed libraries. These constraints should be considered if the application is extended beyond classroom demonstration.

1.12 Chapter Summary

This chapter established the purpose and boundaries of the Personal Expense Tracker. The application focuses on structured transaction entry, local storage, summary calculations, charts, filtering, and export. The next chapter reviews alternative record-keeping approaches and the technical components that make the proposed solution possible.

CHAPTER 2: EXISTING TECHNOLOGIES

2.1 Introduction

A personal expense tracker can be implemented in several ways, including paper-based records, spreadsheet templates, web applications, mobile applications, or a local desktop program. Each approach involves trade-offs between simplicity, accessibility, automation, data control, and development effort. This chapter compares these approaches and explains the technology stack selected for the project.

2.2 Manual Record-Keeping

Paper notebooks are inexpensive and do not require a computer. However, they depend on manual arithmetic and are difficult to filter or chart. If entries are missed or written inconsistently, the user may need to review many pages to prepare a monthly summary. Paper records are therefore suitable for simple personal notes but are less convenient when users need repeated calculations and category comparisons.

2.3 Spreadsheet-Based Tracking

A spreadsheet can calculate totals with formulas and create charts. It can also be customised for a user's categories and preferred reporting periods. Its weaknesses include the possibility of accidentally overwriting formulas, inconsistent data entry, duplicated records, and a lack of guided workflows. A desktop application can reduce these problems by presenting structured fields and predefined operations, although a spreadsheet remains a valid alternative for many users.

2.4 Web-Based and Mobile Trackers

Web and mobile trackers can provide access from multiple devices and may integrate with cloud storage or financial services. Those capabilities require additional design and security work, and may involve external accounts or network access. The project deliberately adopts a local desktop approach so that the implementation can be demonstrated with a compact Python codebase and a local database.

2.5 Desktop Application Approach

A desktop application can place forms, transaction lists, charts, and reports in one program window. Local storage can be convenient for a classroom prototype because it does not require hosting a server. However, local storage also means that users must protect the database file, back it up, and avoid assuming that the data will automatically be available on another device.

2.6 Python

Python is the main programming language used in the project. It supports readable application logic and includes standard-library modules for date handling, CSV processing, filesystem paths, and SQLite access. The code imports modules such as sqlite3, pathlib, datetime, csv, and tkinter. The Python version used for the final demonstration should be recorded separately because the available project archive does not definitively establish the runtime version.

2.7 Tkinter

Tkinter is Python's standard interface to the Tk GUI toolkit. It provides widgets and event-driven interaction for desktop applications. In this project, Tkinter is used to construct the main window and the forms for transaction management, charts, and reports. The interface responds to user actions such as button clicks, text entry, and selection changes.

2.8 SQLite

SQLite is an embedded relational database engine. It stores data in a file and does not require a separate database server process. The project creates a local database at database/expenses.db and uses a transactions table. SQLite is a reasonable fit for a compact single-user project, but the database file should be treated as personal data and backed up appropriately.

2.9 Matplotlib

Matplotlib is a plotting library used to create visual summaries. The project charts include an expense distribution by category, monthly income-versus-expense comparison, and monthly expense trend. A chart can make broad patterns easier to notice than a long list of numbers, but it does not by itself establish why a pattern occurred or provide financial advice.

2.10 OpenPyXL and File Export

OpenPyXL is a Python library for reading and writing Excel workbook files. It is listed in the project's requirements file. CSV is a simple text-based tabular format that can be opened by spreadsheet programs and processed by many tools. Export is useful because it allows the user to preserve a report outside the application, but exported files may contain sensitive financial information and should be handled carefully.

CHAPTER 2: EXISTING TECHNOLOGIES (CONTINUED)

2.11 Comparison of Approaches

Approach	Advantages	Limitations
Paper notebook	Low cost; no software setup	Manual totals; limited search and visualisation
Spreadsheet	Flexible formulas; charts; familiar interface	Formula errors; inconsistent entry; less guided workflow
Web application	Potential multi-device access and central updates	Hosting, network, account, and security requirements
Mobile application	Convenient portable entry; notifications possible	Platform-specific development and device constraints
Local desktop application	Local storage; integrated forms and charts; no server required	Primarily device-bound; backup and file protection are user responsibilities

Table 2.1. Comparison of record-keeping approaches

As shown in Table 2.1, no single approach is best for every situation. The project selects a local desktop application because it offers an achievable balance between structured data entry, local persistence, and visual reporting for a second-year B.Tech mini project.

2.12 Rationale for Technology Selection

The selected technologies fit together with limited infrastructure. Python provides the application logic; Tkinter provides the graphical layer; SQLite provides persistent storage; Matplotlib provides charts; and OpenPyXL supports Excel workbook operations. The use of standard-library modules for common tasks reduces the number of external dependencies. The dependencies listed in requirements.txt are Matplotlib and OpenPyXL.

2.13 Related Technical Concepts

CRUD operations

Create, Read, Update, and Delete are common data-management operations. The tracker supports transaction creation, retrieval, modification, and deletion through its interface and database functions. A clear CRUD workflow helps users correct mistakes without manually editing the database.

Data aggregation

Aggregation converts many transaction rows into useful summaries. Examples include summing income, summing expenses, grouping expense amounts by category, and grouping income and expense totals by month. Aggregated results support the dashboard and charts.

Event-driven interface

A GUI is event-driven: the program responds to actions such as a button click or a change in a filter. Separating event handlers from database functions can improve maintainability and makes it easier to test data operations independently.

2.14 Chapter Summary

This chapter compared common approaches to expense tracking and described the technology stack used in the project. The chosen desktop architecture is appropriate for a local academic prototype, provided its limitations are clearly documented. The next chapter specifies the proposed system and its requirements.

CHAPTER 3: PROPOSED WORK

3.1 Overview of the Proposed System

The proposed system is a modular desktop application that combines transaction management, summary calculation, graphical analysis, and report generation. A user interacts with the Tkinter interface, which invokes the relevant module. Database functions read or modify the SQLite transactions table. The dashboard and charts obtain aggregated values from the database, while the reports interface retrieves records that match selected filters.

3.2 System Architecture

The system can be described as four logical layers: presentation, application logic, data access, and persistent storage. The presentation layer contains the dashboard and screens. Application modules manage expense, income, chart, and report workflows. The data-access functions centralise SQL operations. SQLite stores transaction records in a local file. This logical separation reduces duplication and allows interface components to use shared database functions.

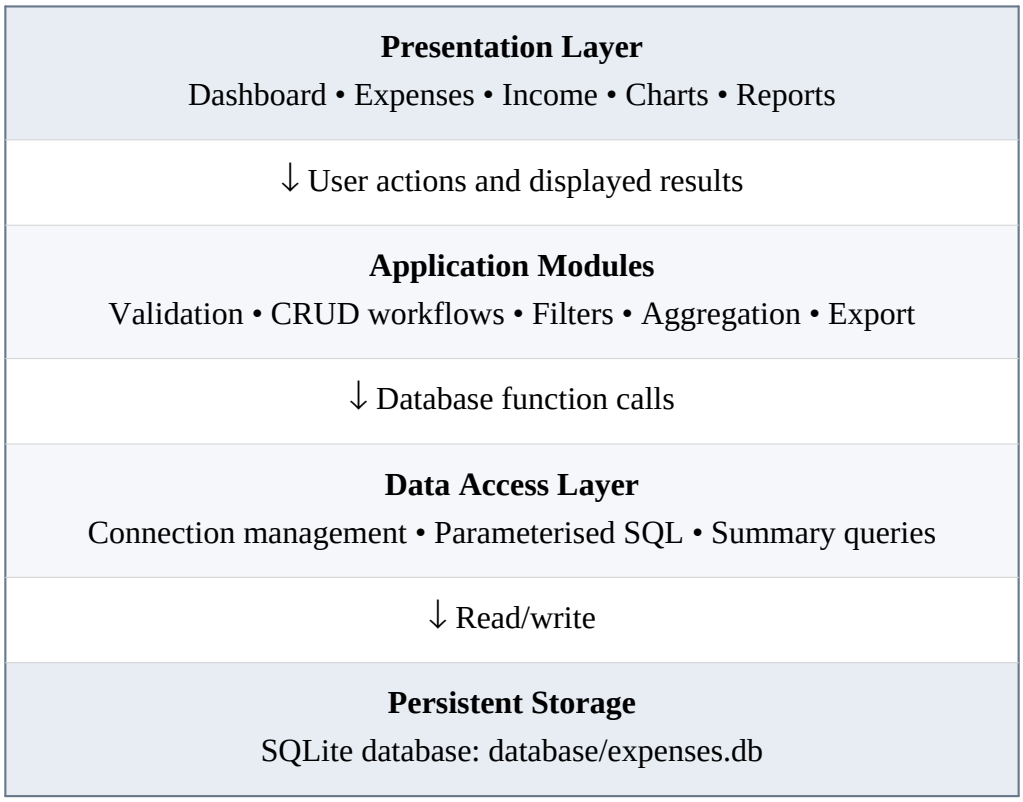


Figure 3.1. High-level architecture of the Personal Expense Tracker

3.3 Functional Requirements

ID	Requirement	Expected behaviour
FR-01	Add transaction	The user can enter transaction details and save a record.
FR-02	View transactions	Stored records can be retrieved and displayed.
FR-03	Edit transaction	An existing record can be selected and updated.
FR-04	Delete transaction	A selected record can be removed.
FR-05	Manage income	Income transactions can be added, viewed, edited, and deleted.
FR-06	Dashboard totals	The interface presents income, expenses, balance, and monthly indicators.
FR-07	Charts	The application displays category and monthly financial visualisations.
FR-08	Filter reports	The user can filter records by date, type, category, and search text.
FR-09	Export reports	Report data can be exported to CSV and Excel, as described in project materials.

Table 3.1. Functional requirements

3.4 Non-Functional Requirements

ID	Quality attribute	Requirement / consideration
NFR-01	Usability	Common tasks should be reachable through clearly labelled controls.
NFR-02	Data integrity	Database operations should preserve transaction fields and identifiers.
NFR-03	Maintainability	Separate modules should make future changes easier to isolate.
NFR-04	Portability	The project should run where a compatible Python environment and dependencies are installed.
NFR-05	Privacy	The local database and exported files should be protected from unauthorised access.
NFR-06	Reliability	Invalid input and database errors should be handled clearly; final testing is still required.

Table 3.2. Non-functional requirements

CHAPTER 3: PROPOSED WORK (CONTINUED)

3.5 Module Responsibilities

Module / file	Responsibility based on the supplied project
main.py	Imports start_dashboard and starts the dashboard when run as the main program.
database.py	Creates the database directory, opens SQLite connections, initialises the transactions table, and exposes transaction and summary functions.
dashboard.py	Builds the main application window and coordinates access to the feature modules.
expenses.py	Provides the expense-management interface and related operations.
income.py	Provides the income-management interface and related operations.
charts.py	Uses Matplotlib to display category distribution and monthly financial charts.
reports.py	Provides report filters, a transaction table, summary cards, and export workflows described in project materials.
requirements.txt	Lists Matplotlib and OpenPyXL as external dependencies.

Table 3.3. Module responsibilities

3.6 User Roles

The reviewed project does not implement a login or role-based access system. The application should therefore be described as a local single-user tracker rather than a multi-user system with separate permissions. Anyone who can access the application and its database file may be able to view or change the stored information, depending on operating-system access controls.

3.7 Input and Output

Input

Typical input includes transaction date, type, category, amount, payment method, and description. The identifier and creation timestamp are generated by the database. Users also provide report filter values, such as a date range, transaction type, category, or search term.

Output

Outputs include the transaction list, dashboard totals, recent transactions, category-level expense chart, monthly income-versus-expense chart, monthly expense trend, and filtered report data. Export files provide a way to preserve or further analyse report information outside the application.

3.8 Data Flow

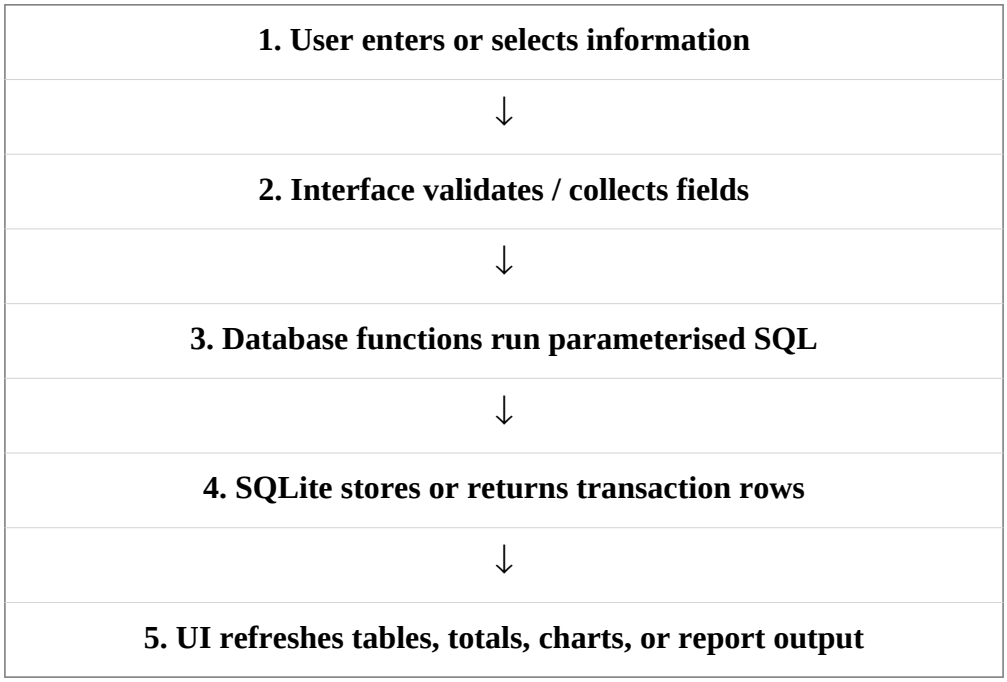


Figure 3.2. Simplified transaction data flow

3.9 Scope Boundaries

The application focuses on user-entered transactions. It does not automatically import bank statements, classify transactions using machine learning, synchronise data with a cloud account, or forecast future expenses. These items may be appropriate future enhancements, but they should not be presented as current functionality.

3.10 Chapter Summary

The proposed design uses a small set of coordinated modules over a shared SQLite database. Functional and non-functional requirements define the expected behaviour and quality considerations. The following chapter describes the data model, development workflow, implementation choices, and testing approach.

CHAPTER 4: METHODOLOGY

4.1 Development Method

The project follows an incremental development approach: identify core requirements, design the transaction structure, implement database functions, build the graphical modules, connect summary queries to the dashboard and charts, and test the main workflows. An incremental approach makes it possible to validate the database layer before relying on it in multiple screens. The report documents the implementation as found in the supplied project archive and does not claim a formal industrial software-development process.

4.2 Requirements Analysis

Requirements were organised around the user tasks visible in the project: maintaining expenses and income, reviewing a dashboard, exploring charts, and generating reports. The database functions support transaction operations and aggregates. The GUI modules provide the entry points for these operations. A final demonstration should confirm each task against the exact code version submitted.

4.3 Database Design

The project uses a single table named transactions. Each row represents one financial transaction. The id field is an integer primary key that is generated automatically. Date, type, category, and amount are required fields. Payment and description are optional text fields with empty-string defaults. created_at records when the database row was created.

Field	Type / constraint	Purpose
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique record identifier.
date	TEXT NOT NULL	Transaction date stored as text.
type	TEXT NOT NULL	Distinguishes income from expense.
category	TEXT NOT NULL	Category or source used for grouping and filtering.
amount	REAL NOT NULL	Transaction amount.
payment	TEXT DEFAULT ''	Optional payment method.
description	TEXT DEFAULT ''	Optional explanatory note.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Database creation timestamp.

Table 4.1. Transaction data dictionary

4.4 Database Connection and Persistence

The database module uses pathlib to create the database directory when needed and sqlite3 to open the database connection. It sets the connection row factory to sqlite3.Row, allowing retrieved values to be accessed by column name. The initialisation function creates the transactions table if it does not already exist. The database file is stored locally at database/expenses.db.

4.5 Database Operations

The database module exposes functions for adding, updating, deleting, and retrieving transactions. It also includes functions for retrieving all transactions, transactions by type, recent transactions, total income, total expenses, balance, current-month expenses, expenses grouped by category, and monthly summaries. This centralisation helps the interface modules reuse consistent database logic.

4.6 Use of Parameterised SQL

The reviewed database module uses parameterised SQL for data-changing operations. Parameterised queries separate SQL instructions from values supplied by the user and are preferable to building SQL statements by concatenating raw input. They help reduce SQL-injection risk and prevent values containing quotation marks from accidentally changing the structure of a query. They do not replace input validation or access control.

4.7 Calculation Logic

The balance is calculated as total income minus total expenses. Category chart data is produced by summing expense amounts grouped by category. Monthly summaries group records by month and calculate income and expense totals for each period. The current-month expense query uses SQLite's strftime function to compare transaction dates with the current year and month. These calculations depend on dates being stored in a consistent format.

CHAPTER 4: METHODOLOGY (CONTINUED)

4.8 Graphical User Interface

Tkinter provides the application window and user interface controls. The dashboard acts as the main entry point and coordinates the expense, income, chart, and report components. The interface is event-driven: user actions trigger the relevant callbacks and refresh routines. Clear field labels, selection controls, confirmation messages, and validation feedback are important for reducing data-entry mistakes.

4.9 Expense and Income Management

Expense and income management share a common transaction structure, while the type field distinguishes the transaction category. The separate screens make the user's task more understandable while allowing shared storage and summary logic. The key workflows are to create a transaction, inspect the saved list, select a record, modify its details, or delete it when necessary. The final test should confirm that changes appear in the dashboard and reports after the interface refreshes.

4.10 Dashboard Processing

The dashboard combines several aggregate values into one overview. Based on the supplied code and project summary, these include total income, total expenses, balance, current-month expenses, a savings-rate indicator, a monthly summary, and recent transactions. Dashboard figures should be understood as summaries of records stored in the local database, not verified bank balances. The formula used for the savings-rate indicator should be checked against the final implementation.

4.11 Chart Generation

The charts module uses Matplotlib and embeds figures in Tkinter with FigureCanvasTkAgg. The supplied project includes an expense-distribution pie chart, a monthly income-versus-expense bar chart, and a monthly expense-trend line chart. The category chart shows how recorded expenses are distributed across categories. The bar chart compares income and expenses by month. The trend chart helps the user inspect changes in expenses over time.

4.12 Report Filtering

The reports module maintains filter variables for the start date, end date, transaction type, category, and search text. The interface includes controls to generate or clear a report and a table with columns for ID, date, type, category, amount, payment method, and description. Filtering can reduce the number of records the user must inspect, but filter correctness should be tested using known sample rows and boundary dates.

4.13 Export Workflow

The project README describes CSV and Excel export. CSV is suitable for simple tabular data exchange, while Excel workbooks can preserve spreadsheet formatting and support further analysis. The final demonstration should verify that the exported file opens correctly, includes the selected report data, uses understandable column names, and does not unexpectedly include records outside the active filters. Exported data should be stored securely.

4.14 Development and Verification Workflow

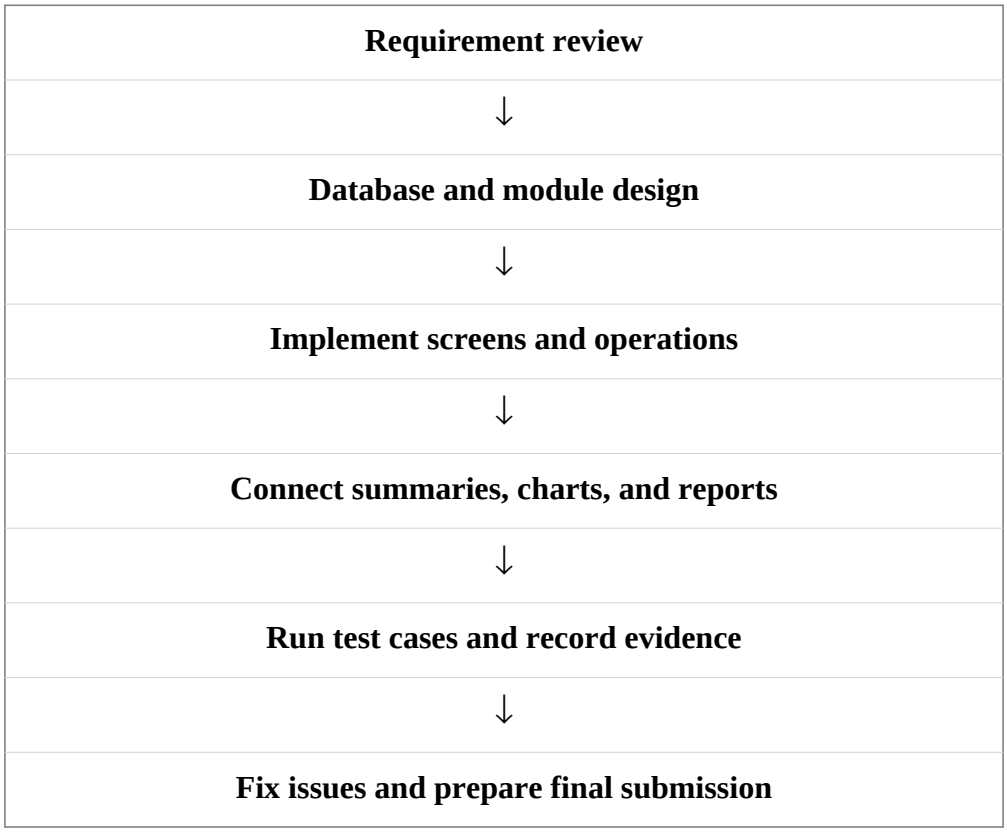


Figure 4.1. Development and verification workflow

4.15 Error Handling and Validation Considerations

A transaction application should reject missing required values, non-numeric amounts, invalid dates, and unsupported transaction types. It should also prevent accidental deletion where possible and show a useful message when a database or export operation fails. The project was only partially tested according to the information supplied; therefore, the report does not assert that every invalid-input case is already handled correctly.

CHAPTER 4: METHODOLOGY (CONTINUED)

4.16 Testing Strategy

Testing should cover individual functions, complete workflows, and integration between modules. Unit-level checks can verify calculations and database operations using a small known dataset. Integration tests can confirm that a saved transaction appears in the correct screen, affects the dashboard totals, is included in charts where relevant, and appears in filtered reports. Export tests should open the generated files and compare their rows with the selected records.

4.17 Proposed Test Cases

Test ID	Action / input	Expected result	Status
TC-01	Add a valid expense	Record is saved and appears in expense list.	To verify
TC-02	Add a valid income	Record is saved and appears in income list.	To verify
TC-03	Edit an existing transaction	Updated values persist and display correctly.	To verify
TC-04	Delete a selected transaction	Selected record is removed from the displayed list and database.	To verify
TC-05	Enter a blank required field	Clear validation feedback; no invalid row saved.	To verify
TC-06	Enter a non-numeric amount	Input is rejected with a clear message.	To verify
TC-07	Choose a date range	Only matching records are shown.	To verify
TC-08	Filter by transaction type	Only the chosen type is shown.	To verify
TC-09	Search for a category or description	Matching records are displayed.	To verify
TC-10	Generate category chart	Chart reflects expense totals grouped by category.	To verify
TC-11	Generate monthly chart	Monthly income and expense values are represented correctly.	To verify
TC-12	Export filtered report to CSV	File opens and contains the expected filtered rows.	To verify
TC-13	Export filtered report to Excel	Workbook opens and contains the expected report data.	To verify
TC-14	Restart application	Previously saved transactions remain available.	To verify

Table 4.2. Test cases and expected outcomes

4.18 Test Evidence and Reporting

For each test, record the test date, environment, input data, expected result, actual result, pass/fail status, and any defect. Screenshots should be captured only after the final code version is run. A test should be marked as passed only when the actual result matches the expected result. Since the project status is partially tested, the table above is a test plan rather than a claim that all tests have passed.

4.19 Reproducibility

A reproducible demonstration should use a known Python environment and install the external dependencies from requirements.txt. The exact Python version, operating system, screen resolution, and dependency versions should be recorded before submission. The available archive contains compiled Python cache files, but these do not reliably establish the intended runtime version.

4.20 Data Protection

The SQLite database may contain personal financial records. The project should be demonstrated with sample data rather than private transactions where possible. The database and exported files should not be published to a public repository if they contain real financial information. A backup should be created before database changes, and the team should confirm what data is included in any shared archive.

4.21 Chapter Summary

This chapter explained the application architecture, database model, calculation approach, charts, report filters, export workflow, and testing strategy. It also identified areas requiring verification, including input validation, export correctness, the savings-rate formula, and the runtime environment. The next chapter analyses the interface evidence and evaluates the implementation without overstating unverified results.

CHAPTER 5: RESULT ANALYSIS

5.1 Introduction

This chapter reviews the implemented features against the project files, README, and supplied interface screenshots. The evidence supports that the application has a dashboard, transaction-management screens, charts, and a reporting interface. Because testing is only partially complete, this chapter distinguishes implemented components from verified runtime behaviour. The screenshots should be checked against the final version before the report is submitted.

5.2 Dashboard

The dashboard is the main entry point of the application. The project summary describes cards or panels for income, expenses, balance, monthly expenses, and savings rate, along with a monthly summary and recent transactions. The dashboard brings frequently used information together. Its accuracy depends on the integrity of the transaction records and the correctness of the aggregation functions.

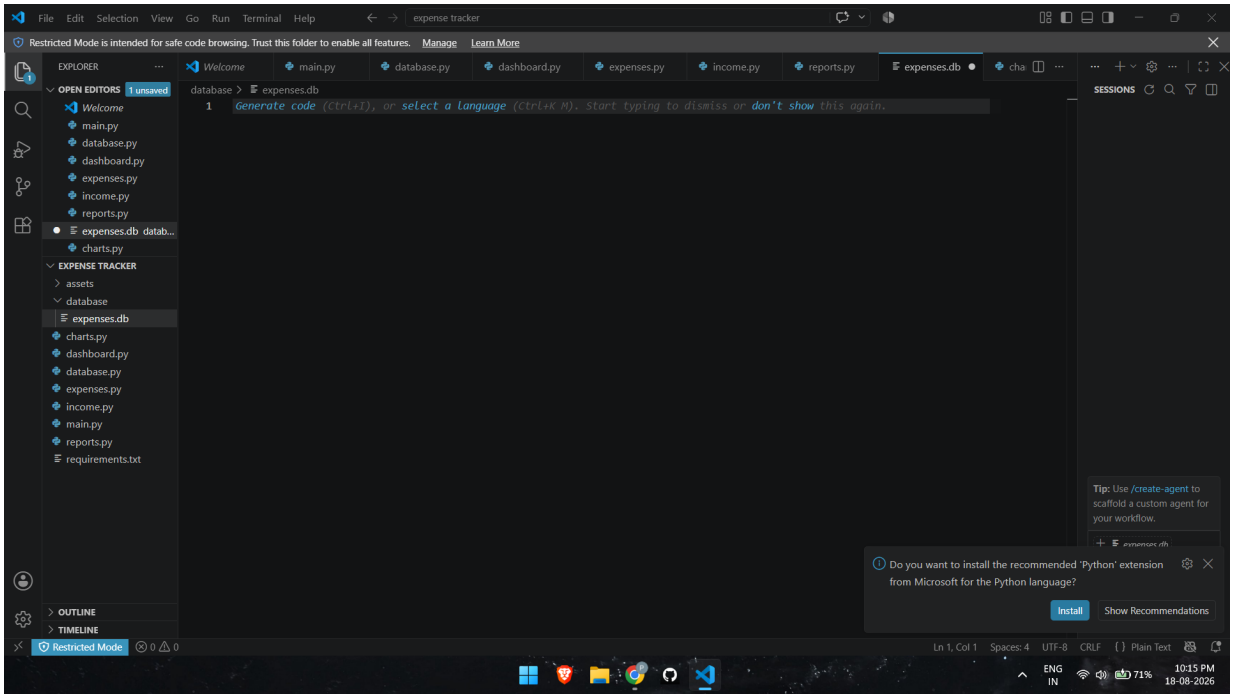


Figure 5.1. Dashboard interface supplied for report preparation

5.3 Expense Management

The expense-management module supports the workflow for maintaining spending records. A useful expense record includes a date, category, amount, payment method, and optional description. The type field distinguishes an expense from income in the shared table. Correct editing and deletion behaviour is important because an incorrect transaction can affect the dashboard, monthly summaries, and charts.

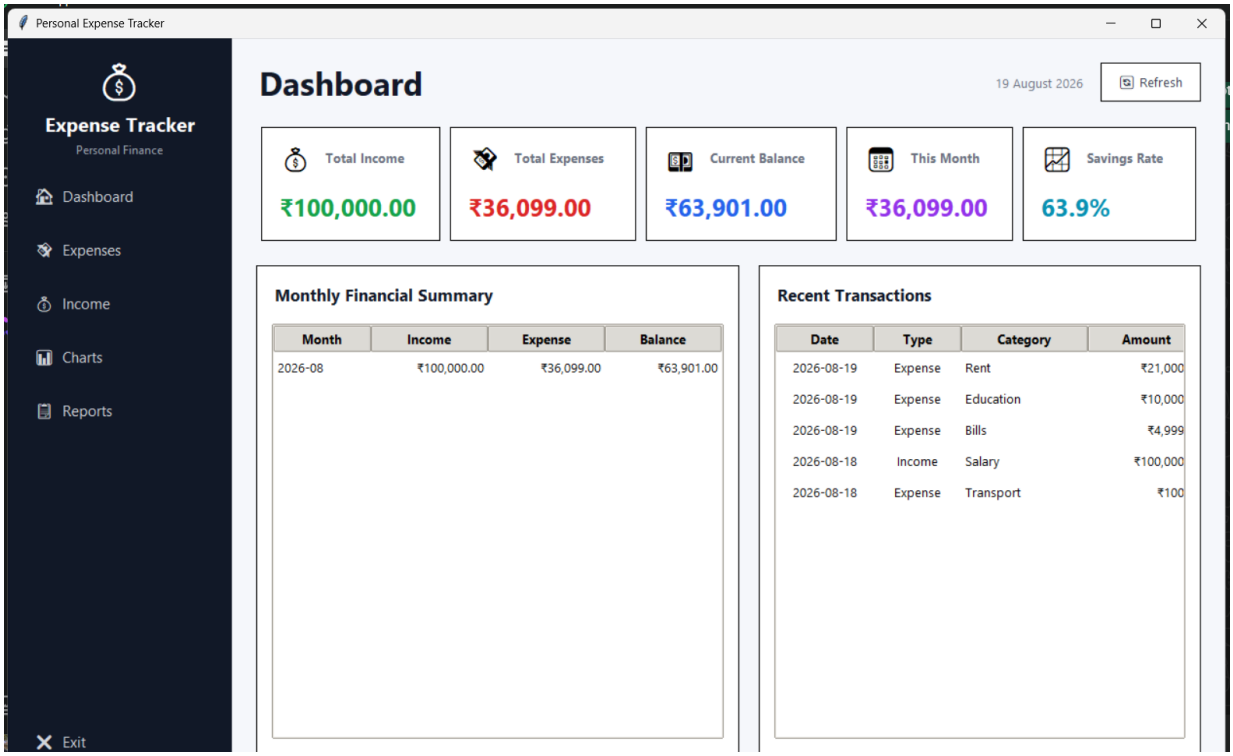


Figure 5.2. Expense management interface supplied for report preparation

5.4 Income Management

The income-management module provides a separate interface for income transactions. Keeping income and expenses in separate screens can make data entry more intuitive while retaining a common underlying schema. The user should be able to correct an income entry and see the revised total and balance reflected in the dashboard after refresh.

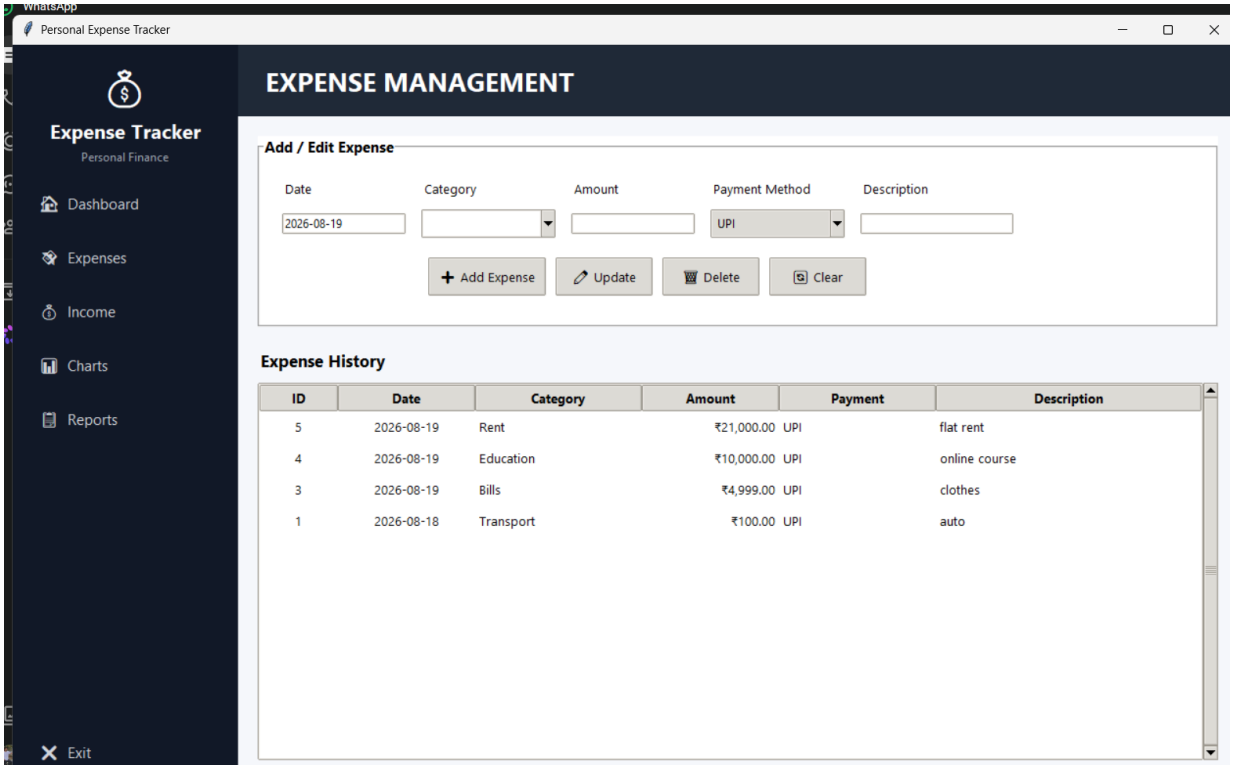


Figure 5.3. Income management interface supplied for report preparation

CHAPTER 5: RESULT ANALYSIS (CONTINUED)

5.5 Financial Charts

The charts module contains three forms of visual analysis: expense distribution by category, monthly income versus expense, and monthly expense trend. These views complement the transaction table by showing relative proportions and changes over time. The charts are descriptive: they summarise recorded data and should not be interpreted as predictions or automated recommendations.

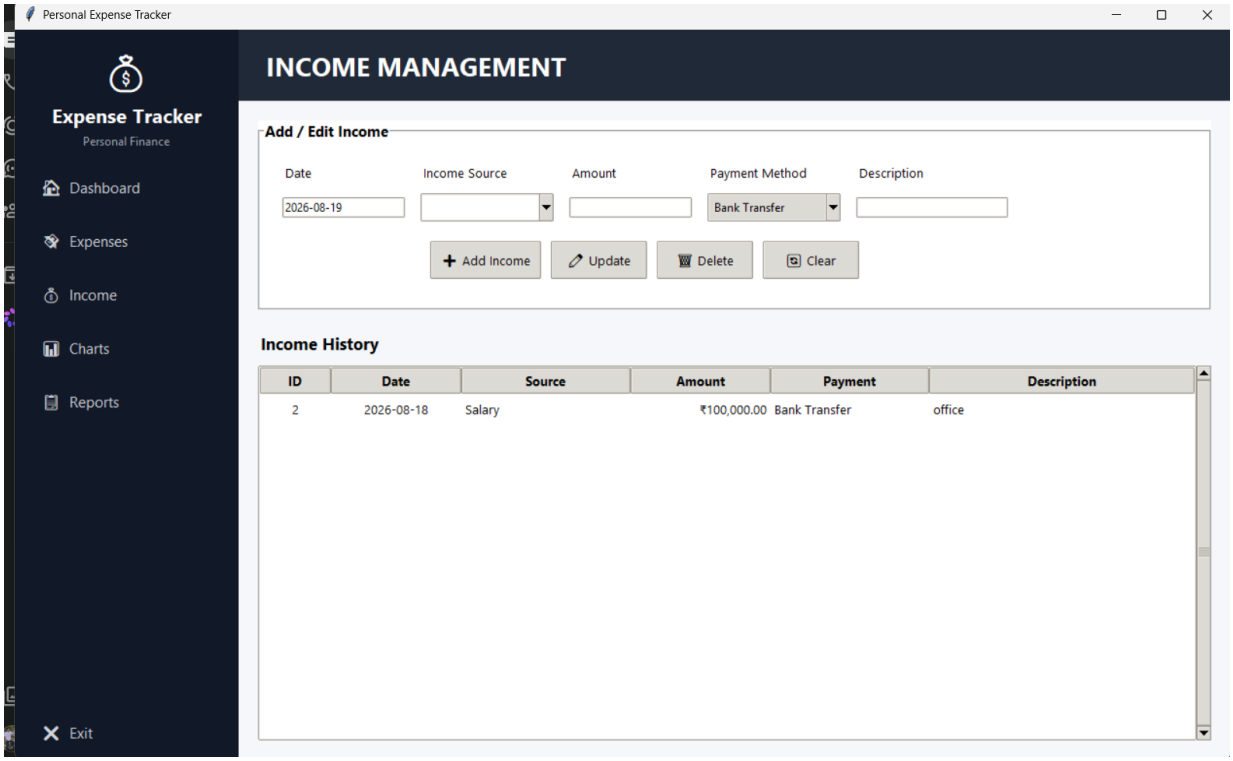


Figure 5.4. Financial charts interface supplied for report preparation

5.6 Reports and Filtering

The reports screen allows a user to narrow transaction data using a start date, end date, transaction type, category, and search text. A report table displays transaction identifiers, dates, types, categories, amounts, payment methods, and descriptions. Summary cards show income, expense, balance, and transaction count. These controls can help the user focus on a particular time period or category.

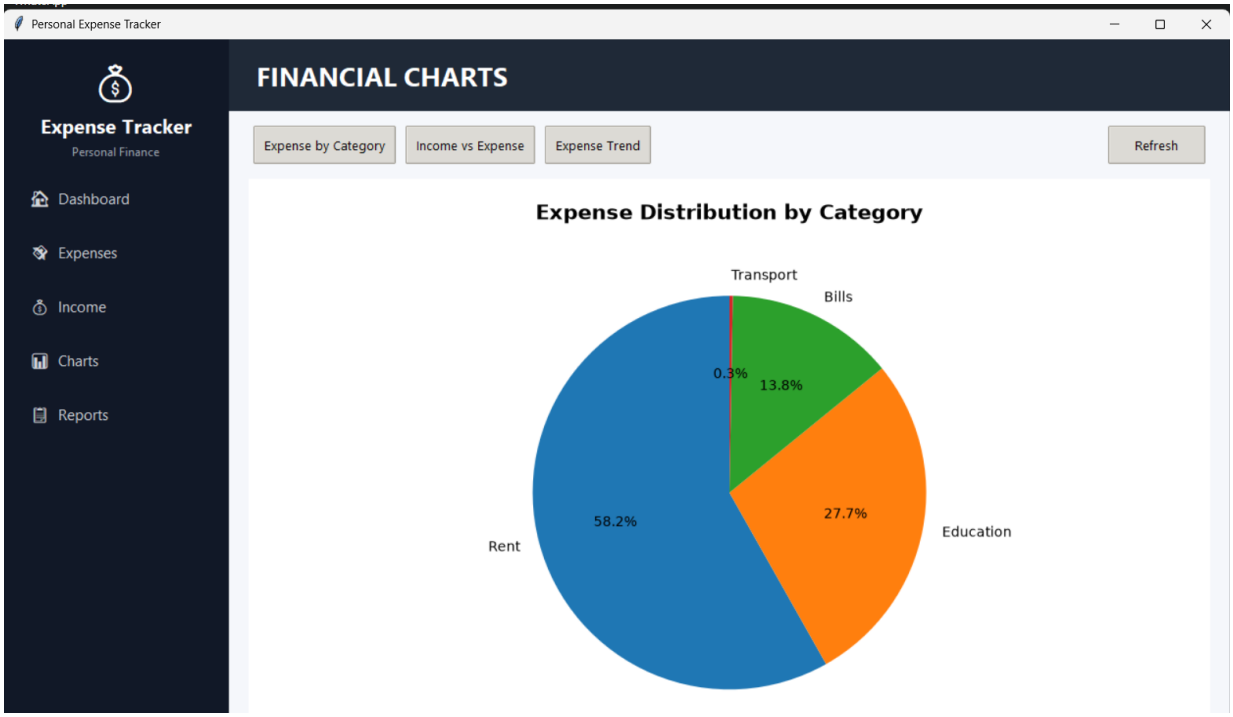


Figure 5.5. Reports interface supplied for report preparation

5.7 Evidence-Based Feature Review

Feature	Evidence available	Evaluation
Dashboard totals	Project code and supplied interface screenshots	Implemented component; verify calculations with known data.
Expense CRUD	Dedicated expenses module and UI screenshot	Implemented component; run create/edit/delete tests.
Income CRUD	Dedicated income module and UI screenshot	Implemented component; run create/edit/delete tests.
Charts	charts.py imports Matplotlib and defines three chart types	Implementation present; verify chart values against database data.
Report filters	reports.py defines date, type, category, and search variables	Implementation present; test filter boundaries.
CSV / Excel export	Project README and OpenPyXL dependency	Documented; verify actual exported output before claiming passed.
Unusual-spending detection	Listed as planned in project summary / README	Future scope only; not implemented.
Forecasting / budget recommendations	Listed as planned in project summary / README	Future scope only; not implemented.

Table 5.1. Evidence-based feature review

5.8 Quantitative Evaluation

A numerical performance evaluation would require a defined dataset and repeatable measurements. Useful metrics include the time needed to add a transaction, time to generate a report, number of records processed, number of successful test cases, and correctness of calculated totals. No benchmark results are claimed in this report because no controlled performance measurements were provided.

5.9 Usability Discussion

The presence of dedicated screens for expenses, income, charts, and reports provides a clear functional grouping. The dashboard reduces the need to open multiple screens for a quick overview. Further usability improvements could include clearer empty-state messages, consistent validation, keyboard navigation, accessibility checks, and confirmation prompts for destructive operations. These improvements should be implemented and tested before being described as existing features.

5.10 Reliability Discussion

SQLite persistence supports retaining transactions between application runs. Parameterised SQL is a positive implementation practice for database operations. Reliability nevertheless depends on error handling, transaction commits, consistent date formatting, and correct UI refresh behaviour. A complete test run is needed to confirm these behaviours across normal and invalid inputs.

CHAPTER 5: RESULT ANALYSIS (CONTINUED)

5.11 Security and Privacy Review

The reviewed project is a local desktop application without a login feature. The absence of authentication means that the application itself does not distinguish between users. Protection therefore depends largely on access to the computer and database file. Real financial data should not be placed in public source-control repositories, and report exports should be stored in locations appropriate to their sensitivity.

5.12 Limitations

Limitation	Impact	Suggested response
Local database file	Data is primarily tied to the computer and file location.	Add documented backup and restore procedures.
No login / roles	The app does not separate user permissions.	Consider authentication only if multi-user use is required.
Partially tested	Some behaviours may remain unverified.	Execute the test plan and retain evidence.
No bank integration	Transactions depend on user entry.	Consider import tools only after privacy and validation design.
No advanced analytics	The app does not automatically detect anomalies or forecast spending.	Treat these as future work and validate any algorithm before use.
Version not confirmed	Environment-specific issues may be harder to reproduce.	Record Python and dependency versions.

Table 5.2. Limitations and risks

5.13 Discussion of Learning Outcomes

The project demonstrates integration across several software-development topics. It uses a relational data model and SQL queries, builds event-driven GUI screens, performs aggregate calculations, and presents data in charts and reports. It also highlights the importance of testing, dependency management, modularity, and data privacy. These learning outcomes align with the goal of a second-year mini project: translating classroom concepts into a working application prototype.

5.14 Chapter Summary

The available evidence supports the existence of the main application modules and visualisation/reporting workflows. The report does not claim that every feature has passed testing. The strongest next step is to execute the proposed test cases on the final version, correct any defects, and replace or supplement screenshots with current evidence. The final chapter summarises the project and identifies future work.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Personal Expense Tracker is a Python desktop application intended to simplify the recording and review of personal income and expenses. It uses Tkinter for the graphical interface, SQLite for persistent local storage, Matplotlib for visual summaries, and OpenPyXL as a dependency for Excel workbook operations. The project is organised into modules for the dashboard, expenses, income, charts, reports, and database operations.

The application structure supports core transaction-management workflows and presents financial information through summary cards, charts, and filtered reports. The shared transactions table stores the key details needed for basic analysis. The modular design provides a foundation for adding more features without redesigning the whole application.

The project is an academic prototype rather than a complete financial platform. It does not implement login, bank integration, cloud synchronisation, or intelligent spending recommendations. Testing is partially complete, so the final submitted version should be checked against the test plan. The report therefore distinguishes code-supported implementation from results that still need empirical verification.

6.2 Objectives Achieved at the Implementation Level

• A Python/Tkinter desktop application entry point is present.

• A local SQLite database and transactions table are defined.

• Dedicated modules exist for expense and income management.

• Database functions support transaction CRUD and financial aggregates.

• The charts module includes category and monthly visualisations.

• The reports module includes filter variables and a transaction table.

• The project documentation describes CSV and Excel export.

The list above reflects implementation evidence in the supplied archive; it should not be interpreted as proof that all workflows have passed runtime testing.

6.3 Future Work

6.3.1 Automated Unusual-Spending Detection

A future version could compare a category's current-period spending with a baseline derived from earlier periods. A simple rule might flag a substantial percentage increase, but the system would need to account for seasonality, missing records, small baselines, and one-time purchases. The feature should explain why an item was flagged and allow the user to dismiss false alarms.

6.3.2 Expense Forecasting

Forecasting could estimate future expenses using historical monthly data. Before displaying predictions, the application should define the required amount of history, select an appropriate model, measure error on held-out periods, and explain uncertainty. A forecast should be presented as an estimate, not a guarantee.

6.3.3 Budget Recommendations

A budget module could allow users to set category-specific monthly limits and compare actual spending with those limits. Recommendations should be transparent and user-controlled. The application should not present general rules as personalised financial advice without a suitable validation and governance process.

6.3.4 Data Backup and Restore

A backup and restore workflow could help users protect their local database. A safe design would warn users before overwriting files, validate the backup, and provide clear instructions for recovery. Export files should be distinguished from complete database backups.

6.3.5 Authentication and Multi-User Support

If the project expands to multiple users, it would need a user model, authentication, access controls, and careful separation of each user's records. Authentication should not be added merely as a decorative login screen; it must be connected to enforceable permissions and a secure data model.

CHAPTER 6: CONCLUSION AND FUTURE WORK (CONTINUED)

6.4 Further Enhancements

Improved input validation

Consistent validation should check required fields, amount format, positive or otherwise permitted values, date format, and allowed transaction types. Error messages should explain how the user can correct an input.

Automated test suite

Automated tests could cover database operations and calculation functions using a temporary database. UI integration tests can then verify that screens display the correct results. Test data should be synthetic and isolated from personal records.

Accessibility and usability

Future interface improvements could include keyboard navigation, consistent spacing, readable contrast, scalable layout, accessible labels, and clear feedback for empty result sets. Usability testing with a small group could identify confusing steps.

Import from bank statements

A future import feature could parse CSV statements, preview rows before saving, detect duplicates, and ask users to map columns. Such a feature should not store bank credentials or upload transaction data without explicit design and consent.

Packaging and installation

A packaged installer could simplify use on computers without a configured Python environment. Packaging should include dependency management, a predictable database location, and documented upgrade and backup procedures.

6.5 Recommendations

• Run the complete test plan and record actual outcomes before final submission.

• Confirm the Python version and external library versions used for the demonstration.

• Verify the formula used for the savings-rate indicator.

• Check CSV and Excel exports using a known filtered dataset.

• Use sample financial data in screenshots and public repositories.

• Keep future analytics clearly separated from the current implemented features.

• Review all names, roll numbers, supervisor details, and institutional formatting before printing.

6.6 Final Summary

The Personal Expense Tracker provides a useful foundation for local personal-finance record keeping and demonstrates practical integration of GUI programming, database operations, charts, and reporting. Its core design is understandable and modular. Completing verification, documenting the environment, and protecting stored data are the most important steps before the project is presented as final.

ENVIRONMENT AND SUSTAINABILITY CONSIDERATIONS

7.1 Introduction

Although the Personal Expense Tracker is not an environmental-monitoring system, its development and use can still be considered from a sustainability perspective. A digital record can reduce dependence on printed transaction logs and can make financial information easier to organise. However, software also depends on electricity, computing equipment, and storage, so the environmental benefit should not be overstated.

7.2 Reduced Paper Use

The application stores records digitally and can display reports on screen. Users may therefore choose to print only the information they need rather than maintaining a paper ledger. This is a potential benefit, not a measured outcome: the project has not conducted a lifecycle assessment or measured paper savings.

7.3 Efficient Local Processing

The application performs basic database queries and chart generation locally. A local design avoids the need for a continuously running application server for the prototype. The actual energy use is not measured, and the report makes no numerical claim about carbon emissions or energy savings.

7.4 Responsible Data Retention

Digital storage can reduce physical paperwork, but records should not be retained indefinitely without a reason. Users should know where the database and exported files are stored and should remove unnecessary copies securely. Backups should be balanced against the need to protect data from loss.

7.5 Accessibility and Inclusive Design

A sustainable software project should be usable by a broad range of users. Future work should consider readable fonts, contrast, clear error messages, keyboard operation, and layouts that work at different screen sizes. The current report does not claim formal accessibility certification.

7.6 Ethical Use of Financial Data

Financial transactions can reveal sensitive information about routines, purchases, and personal circumstances. The project should use synthetic records for public demonstrations, avoid publishing private database files, and clearly communicate that the charts are summaries rather than financial advice. If future analytics are added, users should be able to understand and control how their data is used.

7.7 Sustainability Summary

The application offers a digital alternative to manual record keeping, but no quantified environmental impact has been measured. Its most important sustainability considerations are responsible data handling, reasonable local resource use, maintainability, and inclusive usability.

REFERENCES

[1] Python Software Foundation, “Python 3 Documentation,” Python.org. Available: https://docs.python.org/3/. Accessed: 25 Oct. 2026.

[2] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Python Documentation. Available: https://docs.python.org/3/library/tkinter.html. Accessed: 25 Oct. 2026.

[3] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python Documentation. Available: https://docs.python.org/3/library/sqlite3.html. Accessed: 25 Oct. 2026.

[4] Python Software Foundation, “csv — CSV File Reading and Writing,” Python Documentation. Available: https://docs.python.org/3/library/csv.html. Accessed: 25 Oct. 2026.

[5] Python Software Foundation, “pathlib — Object-oriented filesystem paths,” Python Documentation. Available: https://docs.python.org/3/library/pathlib.html. Accessed: 25 Oct. 2026.

[6] SQLite Consortium, “SQLite Documentation.” Available: https://www.sqlite.org/docs.html. Accessed: 25 Oct. 2026.

[7] SQLite Consortium, “Appropriate Uses For SQLite.” Available: https://www.sqlite.org/whentouse.html. Accessed: 25 Oct. 2026.

[8] SQLite Consortium, “SQL Language Expressions.” Available: https://www.sqlite.org/lang_expr.html. Accessed: 25 Oct. 2026.

[9] Matplotlib Development Team, “Matplotlib Documentation.” Available: https://matplotlib.org/stable/. Accessed: 25 Oct. 2026.

[10] Matplotlib Development Team, “Embedding in Tk,” Matplotlib Documentation. Available: https://matplotlib.org/stable/gallery/user_interfaces/embedding_in_tk_sgskip.html. Accessed: 25 Oct. 2026.

[11] OpenPyXL contributors, “openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files.” Available: https://openpyxl.readthedocs.io/en/stable/. Accessed: 25 Oct. 2026.

[12] Python Software Foundation, “datetime — Basic date and time types,” Python Documentation. Available: https://docs.python.org/3/library/datetime.html. Accessed: 25 Oct. 2026.

[13] Python Software Foundation, “venv — Creation of virtual environments,” Python Documentation. Available: https://docs.python.org/3/library/venv.html. Accessed: 25 Oct. 2026.

[14] Python Software Foundation, “Installing Python Modules,” Python Documentation. Available: https://docs.python.org/3/installing/index.html. Accessed: 25 Oct. 2026.

[15] OWASP Foundation, “SQL Injection Prevention Cheat Sheet.” Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. Accessed: 25 Oct. 2026.

[16] OWASP Foundation, “Input Validation Cheat Sheet.” Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. Accessed: 25 Oct. 2026.

[17] OWASP Foundation, “Logging Cheat Sheet.” Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html. Accessed: 25 Oct. 2026.

[18] NIST, “Secure Software Development Framework (SSDF) Version 1.1,” NIST Special Publication 800-218, Feb. 2022. Available: https://csrc.nist.gov/pubs/sp/800/218/final.

[19] ISO/IEC, “ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” 2023. Available:

https://www.iso.org/standard/78176.html.

[20] G. Booch, J. Rumbaugh, and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Addison-Wesley, 2005.

[21] I. Sommerville, Software Engineering, 10th ed. Pearson, 2015.

[22] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill, 2019.

[23] D. P. Van Rooyen, “The importance of personal financial management,” general personal-finance concepts should be supplemented with an instructor-approved academic source before final submission. [Reference placeholder—replace with a verified source.]

[24] G. L. Bajaj Institute of Technology and Management, “Mini Project Report Format for 2nd and 3rd Year,” departmental format document supplied for this report, 2026.

Important reference note: References [1]–[22] and [24] identify documentation or books relevant to the technologies and software-engineering concepts discussed. Reference [23] is intentionally marked as a placeholder and must be replaced with a verified academic source before submission. Add in-text citations in the final Word/source version wherever these references support specific claims, and verify each web page and access date before printing.

APPENDIX A: FINAL VERIFICATION CHECKLIST

Use this checklist before submitting or printing the report. Mark each item only after personally checking it.

■ Confirm the supervisor's designation and spelling.

■ Confirm the department and university wording required by the college.

■ Confirm all three student names and roll numbers.

■ Run the application from main.py on the machine used for demonstration.

■ Record the exact Python version and installed dependency versions.

■ Test adding, viewing, editing, and deleting both expense and income transactions.

■ Verify total income, total expense, balance, current-month expense, and savings-rate calculations using known sample data.

■ Test date, type, category, and text-search filters using records with known expected results.

■ Export a filtered report to CSV and open it in a spreadsheet application.

■ Export a filtered report to Excel and confirm the workbook contents.

■ Check that the three charts match the underlying transaction data.

■ Replace screenshots if the current interface differs from the supplied images.

■ Ensure the database and screenshots contain no private financial information.

■ Replace the placeholder reference [23] with a verified academic reference.

■ Review the in-text citation placement and figure/table mentions.

■ Update the table of contents and page numbers after any edits.

■ Use Times New Roman, 12-point body text, 14-point bold headings, 1.5 line spacing, and justified paragraphs in the final editable source if required by the department.

■ Print a proof copy and inspect figure clarity, page breaks, and signature spaces.

Appendix B: Known Details to Verify

Item	Current report treatment
Python version	Not confirmed; record the version used for final demonstration.
Supervisor designation	Placeholder retained; confirm before printing.
Testing	Partially tested; proposed test cases are marked 'To verify'.
Advanced analytics	Unusual-spending detection, forecasting, and recommendations are future scope only.
Database privacy	Review database/expenses.db before sharing the project archive publicly.
Reference [23]	Placeholder; replace with a verified academic source.

This PDF is a report draft assembled from the project archive, available screenshots, and the supplied college format. It should be reviewed by the team and supervisor before it is treated as the final submission.